

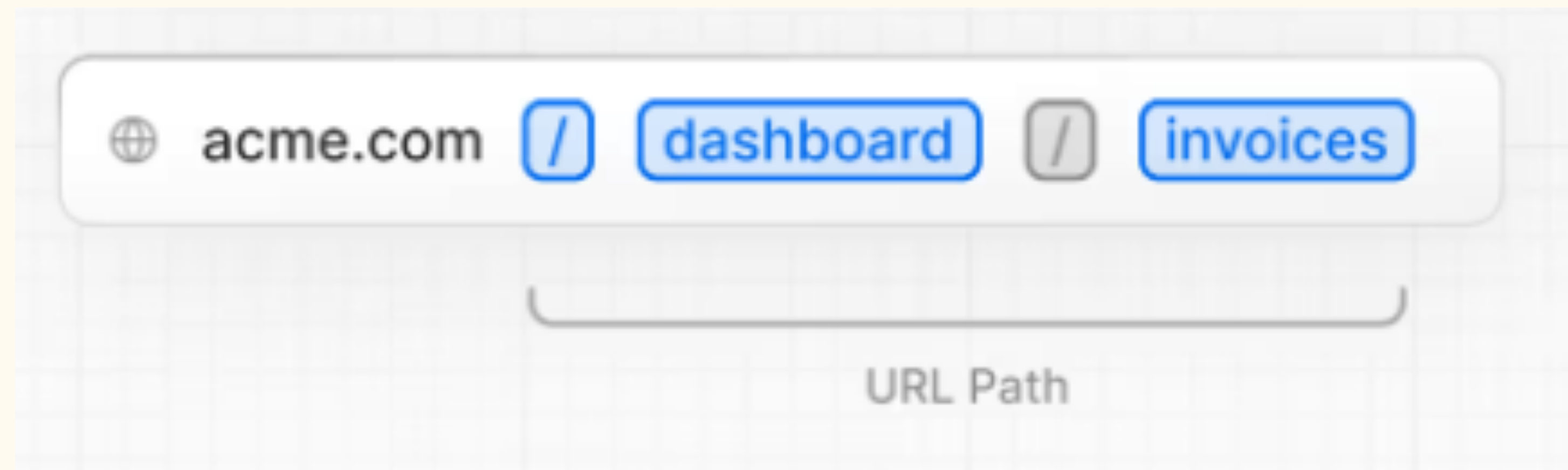
# ROUTING

# I. Lesson

## Definition

Routing in an app is the way the app decides **which page or screen to show** when you click something or go to a certain path, like opening the home page, profile, or settings.





# Benefits of routing

| Benefit                          | Why it matters                                               |
|----------------------------------|--------------------------------------------------------------|
| User-friendly navigation         | People can bookmark/share exact views.                       |
| Deep linking                     | Jump straight to details (e.g., <code>/products/42</code> ). |
| App structure                    | Clear mapping of pages → components.                         |
| Smooth UX (SPA)                  | No full reloads; faster, state is preserved.                 |
| SEO & accessibility (SSR/hybrid) | Pages are indexable; URLs reflect state.                     |
| Browser history                  | Back/forward buttons work as expected.                       |

# Leptos Router

Official routing library for Leptos, designed to enable client-side navigation in single-page applications (SPAs). It provides a declarative way to define routes, handle dynamic paths, and manage navigation without **full page reloads**.



# Leptos Router

**leptos\_router** v0.8.5

Router for the Leptos web framework.

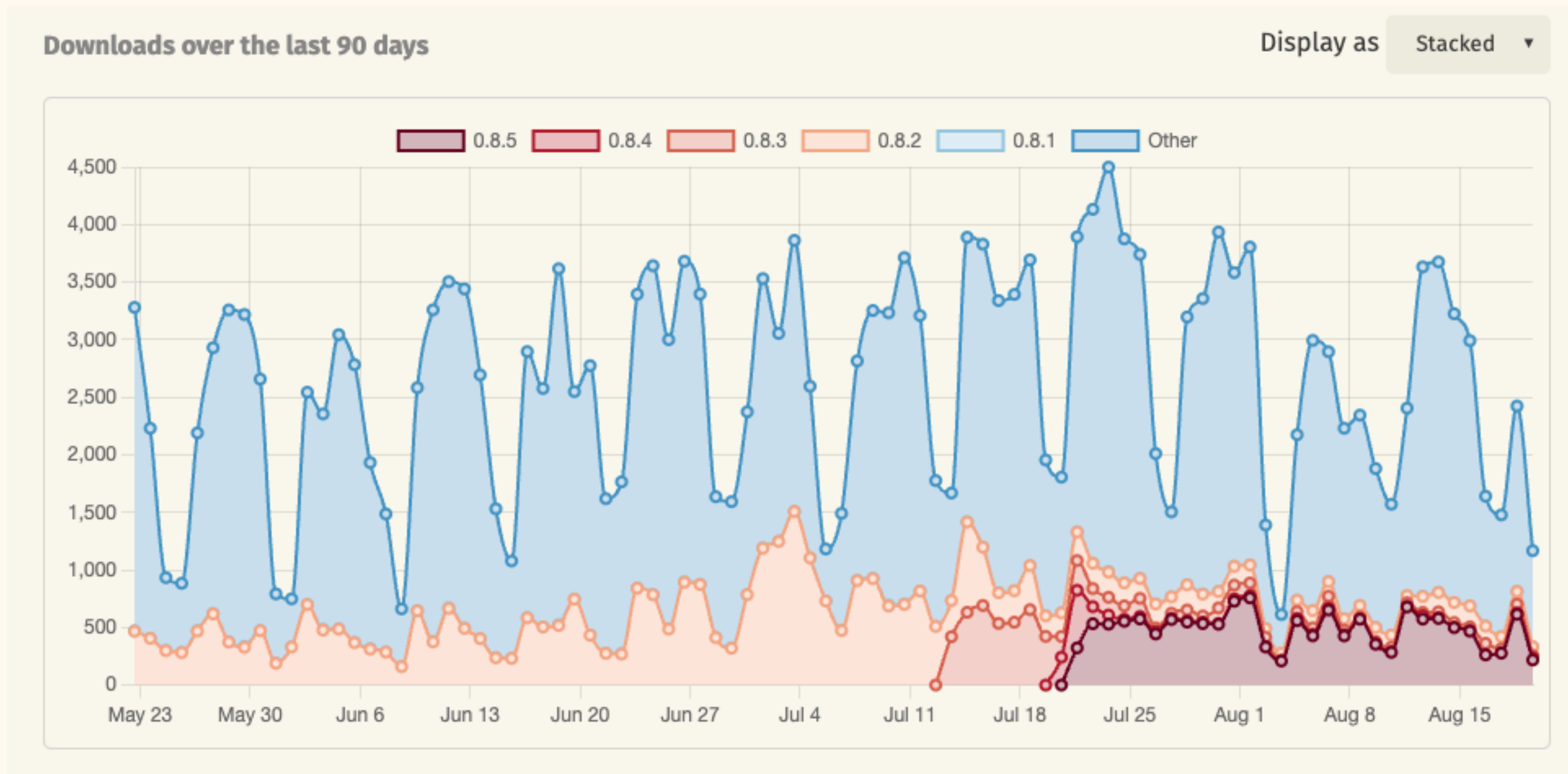
[Repository](#)

📄 All-Time: 1,118,403

📄 Recent: 233,523

🔄 Updated: 21 days ago

# Leptos Router





## II. Exercices

app.rs

```
● ● ●  
  
#[component]  
pub fn App() -> impl IntoView {  
    view! {  
        <Router>  
            <Routes fallback=|| "Page not found.".into_view()>  
                <Route path=StaticSegment("/") view=PageHome />  
                <Route path=StaticSegment("/user") view=PageUser />  
            </Routes>  
        </Router>  
    }  
}
```



```
use leptos_router::{  
    components::{Route, Router, Routes},  
    StaticSegment,  
};
```

Others like path!, ParamSegment, ParentRoute, etc.



# 1. Params



```
http://localhost:3000/user/123 # <- Param
```

app.rs



```
<Routes fallback=|| "Page not found.".into_view()>  
    <Route path=(StaticSegment("user"), ParamSegment("name")) view=PageParam />  
</Routes>
```

# PageParam

```
● ● ●  
  
use leptos::prelude::*;  
  
use crate::utils::param::ParamsUtil;  
  
#[component]  
pub fn PageParam() -> impl IntoView {  
    let params_demo_name = ParamsUtils::extract("name".to_string());  
  
    view! {  
        <div>  
            <h1>"Parameter Page"</h1>  
            <p>"Param: " {params_demo_name}</p>  
        </div>  
    }  
}
```



## 2. Query



```
http://localhost:3000/user?name="Max" # <- Query
```

app.rs



```
<Routes fallback=|| "Page not found.".into_view()>  
  <Route path=StaticSegment("user") view=PageQuery />  
</Routes>
```



# PageQuery

```
● ● ●  
  
use leptos::prelude::*;  
  
use crate::utils::query::QueryUtils;  
  
#[component]  
pub fn PageQuery() -> impl IntoView {  
    let name_query = QueryUtils::extract("name".to_string());  
  
    view! {  
        <div>  
            <h1>"Home Page"</h1>  
            <p>"User name: " {name_query}</p>  
        </div>  
    }  
}
```

Let's code!

Your turn!